SOVEREIGN NET OS

Density · Decay · Convergence

A Technical White Paper

Author: James Chapman / XheCarpenXer

xhecarpenter@gmail.com

Version 1.0 · 2026

Abstract

Sovereign Net OS is a fully decentralised, event-sourced operating system that runs in a single browser tab — no install, no server, no cloud. It is the result of James Chapman (XheCarpenXer) converging four original tools he built over the course of this project: DCX (the foundational deterministic computation model), Planespace (the design system and visual app-building layer), CGK (the Constraint Geometry Kernel — a non-overcommitting distributed capacity primitive via contractive dynamics and lattice-semantic merges, and the event-sourced state machine at the heart of every module), and JSONFlow (the deterministic compilation pipeline from natural language to executable code in nine programming languages).

This paper documents the technical architecture of each tool, how they came together, and the unifying theory that holds the system together: the Density-Decay-Convergence (DDC) model — a physics-inspired framework in which nodes self-organise through excited energy density, natural decay, and emergent convergence. The lava lamp is not a metaphor. It is the architecture.

Contents

1. Origin Story — How Four Tools Became One System

2. DCX — The Deterministic Computation Exchange

3. Planespace — The Design System and App Builder

4. CGK — The Constraint Geometry Kernel

5. JSONFlow — The Deterministic Compilation Pipeline

6. The DDC Model — Density · Decay · Convergence

7. Sovereign Net OS — The Complete System

8. Technical Internals

9. The 11 Canonical Invariants

10. Deployment, Licensing, and Roadmap

1. Origin Story — How Four Tools Became One System

Sovereign Net OS was not designed from the top down. It grew from four tools that James Chapman built independently — each solving a different problem — until the moment they converged into something much larger than any of them alone.

The Problem That Started It

The question James was sitting with was deceptively simple: can software be made fully accountable? Can you build a system where you can take any state it's in, rewind the clock to the beginning, replay every decision, and arrive at exactly the same state? No hidden side effects. No ambient globals. No server you have to trust. Just a log, a function, and determinism all the way down.

The answer to that question required building new primitives — because the existing tools weren't designed with this constraint at their core. What followed was four separate tools, each one necessary, each one building toward the same destination.

The Convergence

DCX provided the theoretical model: entropy enters at the boundary, becomes an event, and from that point everything is deterministic. Planespace gave the system a face — a coherent visual language and an app-building tool. CGK turned the theory into a running kernel: every state change flows through dispatch → record → reduce → applyDelta, in that order, never inverted. JSONFlow solved the last mile: how do you go from a natural-language description of a domain to executable, deterministic code in any programming language — without an LLM making things up in the middle?

When all four converged, the result was a full operating system: identity, messaging, file storage, P2P networking, a code editor with Merkle-DAG versioning, a marketplace, an AI compute node, and a live app builder — running in a single HTML file, in a browser tab, with no backend and no cloud. Every peer is a node. Every tab is sovereign.

2. DCX — The Deterministic Computation Exchange

What It Is

DCX (Deterministic Computation Exchange) is the foundational model from which everything else in the Sovereign OS stack derives. It answers the question: what is the minimum structure needed to make a computation fully accountable and replayable?

The DCX model draws a hard boundary between the outside world and the inside world of a computation. The outside is non-deterministic — user input, network packets, hardware noise, time. The inside is deterministic — once an input enters, its trajectory is fixed.

The Boundary Mechanism

> **Core Invariant** $VM_state_n = deriveState(eventLog[0...n])$

Everything outside the boundary is the host system. Everything inside is a VM. The only path in is through an event — a frozen, immutable, timestamped record of what happened.

Once inside, the event cannot change. Given the same log in the same order, the system always arrives at the same state.

The VM Abstraction

In DCX terminology, a VM is not a hardware virtual machine. It is any execution boundary that satisfies three properties:

1. It has its own event log — an append-only record of everything that has ever happened to it.
2. It has a pure state function — deriveState(log) → state — with no external reads and no side effects.
3. Its outputs (render, network effects) are consequences of state, never inputs to it.

The Lava Lamp as Architecture

James arrived at the lava lamp metaphor to explain why these VMs aren't static — they are alive. A lava lamp contains blobs that rise when heated (excited by energy), cool and sink (decay), and when two excited blobs share the same density they merge (converge). DCX maps this directly to computation:

- Energy: how much primary activity is happening in this VM right now.
- Decay: how long since the last significant event — a measure of staleness.
- Convergence: when two high-energy, low-decay VMs share a causal namespace, they start exchanging events. They do not merge — each keeps its own log — but they begin coordinating.

> **The Insight** Entropy happens outside the VM. It is injected as an event. Once inside, it becomes fixed and replayable. The lava lamp's heat source is external. The blob's behaviour inside the lamp is deterministic physics.

DCX Signal Functions

Every VM in the DCX model exposes three scalar signals, computed purely from its event log:

Signal	Formula	Meaning
energy	count(primaryEvents, window=100) / max(1, total)	Activity rate — how hot is this VM
decay	1 − (eventsSince_lastPrimary / log_length)	Recency — how fresh is its last activity
priority	(energy × w_e) + ((1 − decay) × w_d)	Composite scheduling weight for the router

These are not stored values. They are computed fresh from the log on every read. This is a deliberate design choice: if signals were stored, they would become a second source of truth, violating the core invariant.

3. Planespace — The Design System and App Builder

What It Is

Planespace is James's visual design system and app-building layer — the face that the DCX model needed. If DCX answered the question of how computation should be structured, Planespace answered the question of how the resulting systems should look, feel, and be built by humans.

Planespace is two things in one: a design token system (planespace-theme.js) that provides a consistent visual language across all apps built on the stack, and a visual app builder (app-builder.html) that allows anyone to generate a full event-sourced application from a natural-language description.

The Design Token System

planespace-theme.js is an injected CSS module that applies a complete design language to any web surface. It exposes four dark themes (void, slate, carbon, ash) and six accent palettes (electric, violet, cyan, amber, rose, lime), each specified as CSS custom properties. Every component in the Planespace ecosystem — from the OS shell to generated apps — inherits from this token set.

The type system pairs Syne (display, weight 800) with DM Mono and IBM Plex Mono, creating the distinctive Planespace aesthetic: sharp, technical, high-contrast, and precisely spaced. The theme is applied programmatically via PlaneTheme.apply({ theme, accent }) and can be switched at runtime without a page reload.

The App Builder

The App Builder (app-builder.html) is a 3-stage pipeline that takes a description of an application and produces a complete, self-contained HTML file with DDC invariants baked in. The builder is itself a DDC system:

- The user's prompt is the entropy source.
- Ollama (a local LLM, seed=42, temperature=0) is the deriveState function — deterministic given the same input.
- The hydrated template is the rendered state.
- The download button is the emitted effect.

Every app the builder generates contains a live eventLog, a pure deriveState function, and live signal function computations. Users get DDC compliance for free — they never have to think about it.

The Planespace Shell

The Planespace shell provides a standard OS-like container for built apps: a header with logo and status badge, a sidebar with navigation, a tabbed content area, and a panel system. All built on the token system. Developers building on the stack can apply the shell in one line:

```
<div class="ps-shell">
<header class="ps-header">…</header>
<div class="ps-body">
<nav class="ps-sidebar">…</nav>
<main>…</main>
</div>
</div>
```

4. CGK — The Constraint Geometry Kernel

What It Is

CGK (Constraint Geometry Kernel) is a non-overcommitting distributed capacity primitive that operates via contractive dynamics and lattice-semantic merges. It is the event-sourced state machine that sits at the core of every module in the Sovereign OS stack — the engineering realisation of the DCX model where theory becomes running code.

The geometry is precise: CGK enforces a canonical form for every event, every state transition, and every output. Nothing enters the system without being recorded in exactly one way. Nothing leaves the system without being derived from that record in exactly one way. The contractive dynamics ensure that state transitions are bounded and non-overcommitting. The lattice-semantic merges provide deterministic conflict resolution across distributed nodes.

The Kernel v4 Architecture

The current kernel (Absolute Kernel v4) introduced what James calls "causality inversion" — the key architectural innovation that separates v4 from all prior versions. In v3 and earlier, the log was observational: state changed first, then the change was recorded. In v4, the log is causal: the entry is recorded (frozen and immutable) before any state is written.

This is the difference between a ledger that describes what happened and a ledger that causes what happens. In v4, the log drives state. The pipeline is invariant:

```
External Input (UI / Network / IPC)
↓ SigPipeline.verify(sig) — only signed events proceed
↓ kernel.dispatch(event)
↓ _consume(cost) — metered execution
↓ _record(entry) ← IMMUTABLE — BEFORE any state write
↓ reduce(state, entry) → Delta — pure function, no I/O
```

↓ _applyDelta(delta) ← SOLE WRITE — only path to state

↓ emit effects (IPFS / network / UI / signal update)

The Delta Type

A Delta is the only shape that state-writes may take. A reducer returns a Delta (or null for no-op), and _applyDelta() is the sole write path. The op vocabulary is deliberately small to prevent ad-hoc mutation logic from creeping in:

- set / delete — SharedState key-value
- dagSet / dagEdge — Merkle-DAG nodes and edges
- blockSet / blockDelete / blockPin — CID-addressed block store
- peerRep / peerBan / pubkey — peer reputation and keys
- bw — bandwidth limits
- historyTrim — log compaction

Genesis Modules

The CGK's module system is where the "Genesis" name comes from. A Genesis module is a domain module that declares its event vocabulary, derived state shape, signal functions, invariants, and scheduling contract. Five canonical domain modules ship with the OS:

Module	Primary Events	Priority Weights	Domain
Genesis/identity	IdentityCreated	0.6 / 0.4	DIDs, key rotation, ECDSA P-256
Genesis/messaging	MessageSent	0.5 / 0.5	AES-256-GCM encrypted channels
Genesis/crm	ContactCreated, DealUpdated	0.6 / 0.4	Contacts, deals, activities
Genesis/governance	VoteCast	0.5 / 0.5	Proposals, voting, enactment
Genesis/agent	ToolInvoked	0.7 / 0.3	AI agent runs, tool invocations

The DIK — Deterministic Interaction Kernel

The CGK is complemented by the DIK (Deterministic Interaction Kernel), which extends the same determinism guarantee to UI interactions. In a standard browser application, a click handler directly mutates DOM state — the interaction is non-deterministic and unrecorded. The DIK changes this.

Every pointer interaction (mouse, touch, stylus) is processed through a deterministic frame pipeline: INPUT → UPDATE → LAYOUT → HIT TEST → TARGET LOCK → INTENT EMIT → COMMIT. At each step, the result is deterministic given the inputs. The final intent is converted to a signed kernel transaction by the interaction bridge and dispatched through the kernel's standard path. Every UI action becomes part of the event log. UI sessions can be replayed.

5. JSONFlow — The Deterministic Compilation Pipeline

What It Is

JSONFlow is James's deterministic compilation pipeline — the tool that bridges the gap between a natural-language description of a software domain and executable, type-safe code in nine programming languages, with all DDC invariants guaranteed at generation time.

The key innovation is the separation of concerns across three stages: the LLM produces structured data (Stage 1), a pure compiler converts that data to a language-neutral intermediate representation (Stage 2), and a second pure compiler emits target code (Stage 3). Only Stage 1 involves an LLM. Stages 2 and 3 are completely deterministic — no model, no randomness, no hallucination.

Stage 1 — Genesis (LLM)

The user describes an application in plain language: "build a hotel booking system". This prompt is sent to a locally-running Ollama instance with a precisely engineered system prompt that constrains the output to a canonical Genesis module schema. The LLM operates at temperature=0, seed=42 — maximally deterministic for a given model.

The output is a structured JSON object — a Genesis module — that declares the domain's event vocabulary, derived state shape, signal functions (energy/decay/priority), 11 canonical invariants, reducer contract, scheduler contract, and canonical outputs. The module is then validated against these 8 structural checks before Stage 2 is unlocked:

- derived_state.structure.enforced === true
- emitted_events.structure.enforced === true
- reducers.deriveState present
- signal_functions.energy present
- schedulers.scheduleNextAction present
- All 11 canonical invariants present
- Canonical outputs (ENFORCED_OUTPUTS) present
- Genesis dependencies (Hashing, Addressing) present

Stage 2 — JSONFlow Compiler (Deterministic, No LLM)

Given a validated Genesis module, the JSONFlow compiler (jsonflow.js) deterministically generates five programs in JSONFlow IR — a language-neutral abstract syntax tree expressed as JSON. The five programs are:

4. deriveState — iterates the eventLog and builds full entity state
5. scheduleNextAction — determines the next action from current state
6. computeEnergy — primary event rate over log window
7. computeDecay — staleness since last primary event
8. computePriority — composite scheduling weight

Same Genesis module always produces the same five programs. No LLM. No randomness. The JSONFlow AST uses a small set of typed node primitives:

Primitive	JSON form	Meaning
_set(target, val)	{ type: "set", target, value }	Variable assignment
_get(name)	{ get: name }	Variable read
_getP(obj, prop)	{ get: [obj, prop] }	Property access on object
_if(cond, then, else)	{ type: "if", condition, then, else }	Conditional branch
_foreach(item, iter)	{ type: "foreach", item_var, iterable, body }	Iteration
_cmp(l, op, r)	{ compare: { left, op, right } }	Comparison expression
_add/_sub/_mul/_div	{ add: [a, b] } etc.	Arithmetic
_ret(val)	{ type: "return", value }	Return value

Stage 3 — Code Compiler (9 Languages)

The final stage takes any JSONFlow program and compiles it to executable code in the target language. Nine targets are supported out of the box: Python, JavaScript, TypeScript, Rust, Go, Java, C#, Swift, and Kotlin.

Each language has a configuration object in lang-configs.js that specifies every syntax-specific detail: indentation string, statement terminator, assignment operator, if/else/while/foreach syntax, block close character, return statement form, assert form, log form, and comment prefix. Type mapping (types.js) covers the full matrix — JF_TYPES[jsonType][language] and JF_DEFAULTS[jsonType][language] — for all primitive types: string, integer, number, boolean, array, object.

The compiler handles nested structures, property access (with language-specific Map/object syntax for Java, Kotlin, C#, Rust, and Go), object literals, string concatenation, boolean operators, and fallback to well-labelled TODO comments for unsupported nodes.

End-to-End Test test_e2e.js runs the full Stage 1→4 pipeline in Node.js with a hotel booking module, executes the compiled JavaScript deriveState against 5 synthetic events, and asserts on 9 output values. All 9 pass.

6. The DDC Model — Density · Decay · Convergence

The DDC model is the unifying theory that holds the four tools together. It is not a separate product — it is the lens through which DCX, Planespace, CGK, and JSONFlow are all interpretable as parts of a single coherent system.

The Five-Layer Reduction

Any computation in the DDC model can be reduced to five layers:

Layer	DDC term	What it is
Host system	Physical machine	Non-deterministic. Source of entropy. Exists outside all VMs.
VM (Module)	Execution boundary	Deterministic runtime. Isolated. eventLog is its only source of truth.
eventLog	Memory	Append-only, immutable. The VM's only input to deriveState.
deriveState	CPU	Pure function. No I/O. Same log → same state. Always.
render/effects	Outputs	DOM, network, metrics. Never feed back into state directly.

How Nodes Find Each Other

In the DDC model, VMs do not have a directory. They do not register with a central authority. They find each other by physics: two VMs with overlapping correlation namespaces and matching high-priority signals will be routed together by the event router. This is convergence.

The convergence condition is:

```
converge(VM_a, VM_b) iff:
priority_a > θ — VM_a is active enough
AND priority_b > θ — VM_b is active enough
AND overlap(corrSpace_a, corrSpace_b) — they share causal ancestry
```

θ (theta) is the convergence threshold, typically 0.5. When convergence is satisfied, the router begins delivering events from VM_a to VM_b's log, and vice versa. Each VM still maintains its own log. They do not merge — they coordinate.

Causal Traceability

Every event in the system carries two IDs that make the full causal graph reconstructible at any point:

- correlationId — identifies the originating logical operation. The same string appears on every event causally linked to a single user action, across any number of VMs and any number of hops.
- causationId — the id of the specific event that directly caused this one. Following causationId links lets you trace the exact path from any state back to the external input that triggered it.

7. Sovereign Net OS — The Complete System

What It Is

Sovereign Net OS is a fully decentralised operating system that runs in a single browser tab. It combines every capability James built into one self-contained HTML file: P2P mesh networking, a Merkle-DAG versioned code editor, end-to-end encrypted messaging, a decentralised social feed, CID-addressed file storage, peer video streaming, a drag-and-drop app builder, live network visualisation, a security auditor, local AI inference, and a 30+ command terminal.

No DNS. No CDN. No backend. No API key. Open index.html. You are the node.

The P2P Mesh

Peer discovery in the browser uses BroadcastChannel — the same-origin pub/sub primitive that allows multiple tabs on the same machine to communicate directly. Each tab announces its DID (Decentralised Identifier), public key (ECDSA P-256), handle, and timestamp on the shared bus. Tabs that hear the announcement record the peer and begin exchanging signed gossip messages.

In Electron mode (with a live Kubo IPFS daemon), BroadcastChannel is replaced by WebSocket for cross-machine connectivity, and all simulated subsystems (CIDs, bandwidth, pubsub) are replaced with real IPFS operations. The browser and Electron modes use identical application code — only the transport and storage layers differ.

Identity and Cryptography

Every node in the Sovereign OS mesh has a unique DID generated from an ECDSA P-256 keypair on first load. The DID format is did:sos:<hex-pubkey>. Every event, commit, message, and file operation is signed with this key. The SigPipeline module verifies all incoming signatures before they are admitted to the kernel.

Channel encryption uses AES-256-GCM symmetric keys, shared peer-to-peer via invite links. Private channels generate a new 256-bit key on creation; the key is encoded into a base64 invite string and shared out-of-band. Only peers with the key can decrypt messages.

The Merkle-DAG Code Editor

The code editor is backed by a Merkle-DAG versioning system — a content-addressed directed acyclic graph of commits, each containing a diff, parent CID, author DID, ECDSA signature, and vector clock. Every save creates a new immutable commit node. The CID of a commit is the SHA-256 hash of its content, so two nodes with the same CID are guaranteed to have the same content.

Conflict resolution uses deterministic LCA (Lowest Common Ancestor) merge with LWW-CRDT (Last Write Wins, Conflict-free Replicated Data Type) fallback. Real-time collaboration is implemented through cursor sharing — peers broadcast signed cursor positions, and each editor renders remote cursors as coloured overlays with peer handles.

The Storage Stack

Files are stored in an IndexedDB-backed block store with SHA-256 content addressing. Every stored block is accessible by its CID (Content Identifier). In Electron mode, the block store is replaced by a live Kubo daemon — files stored in Sovereign OS are immediately addressable from any IPFS gateway.

The AI Compute Node

Sovereign OS includes a local AI inference node that connects to Ollama running on the user's machine. The AI node runs with no cloud dependency — all inference is local. The same Ollama connection powers the JSONFlow pipeline's Stage 1 (Genesis module generation) and the App Builder (app generation from schema).

8. Technical Internals

Web Platform Primitives

Sovereign OS uses only Web platform primitives — no npm packages, no build step, no transpilation. The specific APIs used:

API	Used for
BroadcastChannel	Real multi-tab P2P mesh (same-origin peer discovery and messaging)
IndexedDB	Persistent, structured local storage for all state
Web Crypto / SubtleCrypto	ECDSA P-256 signing, AES-256-GCM encryption, SHA-256 hashing, PBKDF2 key derivation
ReadableStream	Streaming AI inference output from Ollama
WebRTC DataChannel	Direct peer-to-peer data connections (simulated in browser, real in Electron)
Canvas 2D	Network visualisation and node graph rendering

File Structure

The project is organised into three distinct layers:

- DDC/ — top-level: the ddc-infrastructure-spec.html reference document

- DDC/Sovereign-Net-O.S.-main/ — the runtime OS (index.html + src/)
- DDC/genesis-jsonflow/ — the compilation pipeline (index1.html, app-builder.html, js/, schemas/, jsonflow/)

The DDC Compliance Layer

The Sovereign OS index.html ships with a DDC compliance layer injected before the closing body tag. This layer adds MODULE_SCHEMAS for all five canonical domains, per-module append-only event logs, pure computeEnergy/computeDecay/computePriority functions, a DDCKernel.dispatch() function that enforces log-before-state, and five new terminal commands: modules, signals, module-log, derive-state, and ddc-dispatch.

The compliance layer also patches the existing EventBus.emit() to stamp correlationId and causationId on every event, and routes MSG events to the messaging module log and IDENTITY events to the identity module log automatically. Existing OS functionality is entirely unaffected — the layer adds to the surface, never overwrites it.

Performance Considerations

The kernel's sandbox for code execution enforces a 50,000 operation limit and a 3-second wall-clock timeout. Blocked globals prevent untrusted code from accessing fetch, XMLHttpRequest, WebSocket, localStorage, sessionStorage, indexedDB, document, window, parent, and top. Output is captured and rendered in an inline console panel.

Signal functions are computed on every EventBus.emit() call and on every DDCKernel.dispatch() call. They are O(n) in the length of the event log. In practice, logs rarely exceed a few hundred events per session, keeping computation under 1ms. Log compaction (historyTrim Delta op) is available for long-running nodes.

9. The 11 Canonical Invariants

Every Genesis module must declare all 11 of these invariants. They are enforced by the schema validator (state.js:validateModule) and by the kernel dispatch pipeline. These are the rules that define what it means to be a DDC-compliant system:

#	Invariant
1	Replay(state) == Live(state)
2	No hidden or mutable state outside the log
3	All outputs traceable to explicit events
4	Signals are functions of history, not stored values
5	Same input sequence → identical execution path
6	Same event log → identical emitted event sequence
7	Every event has a deterministic total ordering key
8	Reducers are pure functions of (state, event)
9	Schedulers are pure functions of event history with deterministic tie-breaking

#	Invariant
10	Signal functions are pure functions of event history
11	Given the same event log, emitted event sequence is identical

10. Deployment, Licensing, and Roadmap

Quick Start

Browser (no install):

9. Download DDC/Sovereign-Net-O.S.-main/index.html
10. Open in Chrome, Edge, or Firefox
11. Open 2–4 tabs — they auto-discover each other and form a live mesh

Electron + real IPFS:

12. Run: bash Sovereign-Net-O.S.-main/scripts/setup.sh
13. Run: npm start

Genesis/JSONFlow pipeline:

14. Ensure Ollama is running locally: ollama serve
15. Open DDC/genesis-jsonflow/index1.html
16. Describe an app: "build a task management system"
17. Stage 1 (LLM) → Stage 2 (JSONFlow) → Stage 3 (pick language, compile)

License

Sovereign Net OS and the DDC Infrastructure are dual-licensed by James Chapman (XheCarpenXer):

- Community License (LICENSE-COMMUNITY): free for personal use, open-source projects, academic research, and evaluation. Government use and commercial use by for-profit entities are not permitted under this license.
- Commercial License (LICENSE-COMMERCIAL): required for commercial deployment, government use, or any use not covered by the Community License. Contact xhecarpenter@gmail.com for terms.

All source files carry the following header:

> Copyright (c) 2026 Sovereign OS Contributors
>
> Licensed under the Sovereign OS Community License (LICENSE-COMMUNITY).
>
> Commercial use requires a separate Commercial License (LICENSE-COMMERCIAL).

Core invariant: VM_state_n = deriveState(eventLog[0…n])

Roadmap

The following are open development directions for the project:

- Unified schema layer — merge genesis-jsonflow/schemas/*.js with the inline MODULE_SCHEMAS in Sovereign OS so there is one canonical source for all domain schemas.
- Module deployment pipeline — automated path from Stage 3 compiled module code to a running Sovereign OS VM, without manual copy-paste.
- Cross-VM event routing (full DDC router) — implement the convergence condition (priority threshold + correlation overlap) in the OS mesh layer.
- App Builder → OS module — make apps generated by the App Builder mountable as live Sovereign OS modules.
- Rust/WASM kernel — port the CGK (Constraint Geometry Kernel) to Rust compiled to WebAssembly for performance-critical deployments.
- Mobile clients — Progressive Web App packaging for iOS and Android.

This white paper was written by James Chapman (XheCarpenXer).

Contact: xhecarpenter@gmail.com

The system described in this paper — DCX, Planespace, CGK, JSONFlow, and Sovereign Net OS — was designed and built entirely by James Chapman. The core invariant that underlies every part of it:

VM_state_n = deriveState(eventLog[0…n])

Nothing else is allowed to influence state.